

Quiz 4

Filename: q4.c

Instructions

Use the provided starter code, which you can access [here](#), and copy it in its entirety. This file is intentionally incomplete to simulate a written exam. Further, you will not be provided the contents of the q4.h file. You are strongly encouraged to first work out your solutions using pencil and paper. Once you are confident in your answers, type the final solutions into this file. You may only write code inside the bodies of the required functions. Do not modify function signatures, `#include` directives, or any other part of the file. Write your answers using a plain text editor only. Do not use a compiler, debugger, or IDE while completing this file. Save your file as q4.c. You may submit multiple times on Gradescope before the submission deadline. Only the final submission made before the deadline will be graded.

Important: Certain compiler warnings are treated as errors, and any such warning will cause compilation to fail. If your code does not compile on Gradescope, your submission will receive a score of 0. All successfully compiled submissions will be evaluated for both correctness and proper implementation.

Important: Your score may be adjusted upon manual inspection if the required data structure (e.g., stack, queue) is not used as specified, even if the autograder reports full credit. All submissions are subject to manual review.

Exam Preparation Tip

As you work through the questions, time yourself. You should aim to complete each question within 10 minutes.

TODO 1 (20 points)

Consider the following function:

```
void foobar(void)
{
    int i, n = 9;    // There are 9 elements in the array.
    int array[] = {3, 18, 58, 23, 12, 31, 19, 26, 3};

    Stack *s1 = createStack();
    Stack *s2 = createStack();
    Queue *q = createQueue();

    for(i = 0; i < n; i++)
        push(s1, array[i]);

    while(!isEmptyStack(s1))
    {
        while(!isEmptyStack(s1))
            enqueue(q, pop(s1));        // Pop elem from s1 and enqueue it to q

        while(!isEmptyQueue(q))
            push(s2, dequeue(q));        // Dequeue from q and push onto s2

        printf("%d ", pop(s2));          // Pop from s2 and print element

        while(!isEmptyStack(s2))
            push(s1, pop(s2));          // Pop from s2 and push onto s1
    }

    freeStack(s1);
    freeStack(s2);
    freeQueue(q);
}
```

What is the **exact output** produced by this function? You may assume that all referenced functions (such as `createStack()`, `createQueue()`, `push()`, `pop()`, `enqueue()`, `dequeue()`, etc.) are implemented correctly and behave as expected.

TODO 2 (40 points)

We will simulate a simple battle game involving players arranged in a queue. These are the rules:

- The game consists of multiple rounds. In each round, the two players at the **front** of the queue battle each other.
- The winner is the player with the higher hp. If both players have the same hp, the **second** player in line wins the round.
- The loser is permanently removed from the game. The winner is added to the **back** of the queue.
- The game continues until only one player remains in the queue.

Complete the function `battle_game` that simulates this process. It must print the name of the overall winner. Do not add any trailing whitespaces. You are provided with the following structure definitions and helper functions that you must use.

```
typedef struct Player_s {
    char *name;
    int hp;
    struct Player_s *next;
} Player;

typedef struct Queue_s {
    Player *front;
    Player *back;
    int size;
} Queue;

void enqueue(Queue *q, Player *p);    // Enqueues player p to the queue q
int size(Queue *q);                  // Returns the number of elements in q
Player *dequeue(Queue *q);           // Removes front node, returns ptr to it
Player *front(Queue *q);              // Returns pointer to front node
void delete_player(Player *p);        // Frees memory pointed to by player
```

Note: It is guaranteed that the queue pointed to by `gameQueue` contains at least one player.

```
void battle_game(Queue *gameQueue) {

}
```

Important Note

After the overall winner has been determined, ensure that all allocated memory is properly freed.

TODO 3 (40 points)

Consider a string representing an arithmetic expression that may contain two types of parentheses: '(' and '{'. An expression is considered **imbalanced** if:

- Parentheses are not closed in the correct order,
- There are extra opening parentheses, or
- There are extra closing parentheses.

The following table shows several examples and their corresponding status.

Example	Status
({})	Imbalanced due to incorrect order of }
((	Imbalanced due to extra (
{()}	Balanced
({)})	Imbalanced due to extra)
{}	Balanced

Complete the function that accepts an expression as a parameter and returns 1 if the expression is balanced; otherwise, it returns 0. Your solution **must use stack operations** to determine whether the parentheses are properly matched.

Assume that the stack structure and its associated operations have already been implemented and are available for use. You may also assume that the stack has sufficient capacity, so you do not need to check for overflow during this process.

```
void initialize(Stack *s);           // Initializes an empty stack (does not allocate)
int push(Stack *s, char value);      // Pushes the char value onto the stack
int is_empty(Stack *s);              // Returns 1 if the stack is empty, 0 otherwise
char pop(Stack *s);                  // Pops and returns the character at the top
char peek(Stack *s);                 // Returns the character at the top
```

Note: pop() and peek() return 'I' if the stack is empty.

```
int is_balanced(char *exp) {
```

```
}
```

Important Note

You may assume that exp is a non-empty string. It contains only the following characters: '(', ')', '{', and '}'. You are not allowed to use the `string.h` library. Therefore, you must determine the length of the string without calling functions such as `strlen`. **Hint:** In C, strings are null-terminated, meaning they end with the special character `'\0'`.